Position Embeddings Exploration

Part 1a is on latex.

Question 1A

ii) (1 point) Think about the implications of the result you derived in part i. Explain why this property of the Transformer model could be problematic when processing text.

- This permutation property of the transformer model is problematic when processing text without any adjustments because a transformer should be able to differentiate between tokens based on their position in the sequence. If the tokens can be permuted such that they appear in a different order, the transformer should change its output accordingly, but this current version of the transformer which leads to the output Z_perm for any permuted input X_perm being Z_perm = PZ is invariant to these positional differences. With this invariance, the model fails to learn sequence-dependent representations of the input text, which is a significant drawback.

Question 1B

a) Position embeddings.

i) Do you think the position embeddings will help the issue you identified in part (a)? If yes, explain how and if not, explain why not.

- The positional embeddings will definitely help the issue we identified in part a. The reasoning behind this is that every token in the sequence will now have a unique sine and cosine value signature added to it. Based on how the positional embeddings are added to the original input, where the embeddings are a matrix of high-period periodic functions capable of capturing the relative order of the tokens, the math such that the output Z_perm for any permuted input X_perm becomes Z_perm = PZ is no longer valid. The reason this is no longer valid is because the positional embeddings are fixed and depend on the original positions of the tokens, so permuting the input also permutes the positional embeddings, and P(X + pos_emb) is not equal to PX + pos_emb. Therefore, the output is no longer just a permutation of the original output, and the model can distinguish between different token orders.

ii) Can the position embeddings for two different tokens in the input sequence be the same? If yes, provide an example. If not, explain why not.

- Yes, position embeddings for two different tokens can be the same in theory due to the periodic nature of sine and cosine functions. For example, if the embedding dimension d=2, then the position embedding at position t is just [sin(t), cos(t)], which repeats every 2pi. Therefore, positions t = 0 and t = 2pi would have the exact same embedding. However, in practice, models use large d (i.e. d = 512), with different frequencies across dimensions, making it extremely unlikely that two different positions will share the same embedding within any realistic sequence length. Essentially, the position embeddings for two different tokens will realistically never happen if the positional embeddings are implemented well

different frequencies, dimensionality); it would take a really, really long time for the EXACT same sine and cosine function inputs to appear for a given token.

2. Pretrained Transformer models and knowledge access

(2d) Make predictions (without pretraining).

Report your model's accuracy on the dev set (as printed by the second command above). We also have Tensorboard logging in assignment for debugging.

Dev set Accuracy:

```
number of parameters: 3323392
Model on device:  cuda:0
500it [00:50,  9.99it/s]
Correct: 8.0 out of 500.0: 1.6%
2025-04-09 23:02:19.934538: I tensorflow/core/util/port.c
2025-04-09 23:02:19.952008: E external/local_xla/xla/stre
WARNING: All log messages before absl::InitializeLog() is
```

London Baseline:

```
PS C:\Users\micah\OneDrive\Gmail OneDrive\Desktop\CMU\_Spring 2025\Deep Learning\HW
5\true_student> & C:/Users/micah/anaconda3/envs/clinical_ai/python.exe "c:/Users/mi
cah/OneDrive/Gmail OneDrive/Desktop/CMU/_Spring 2025/Deep Learning/HW5/true_student
/src/london_baseline.py"
500it [00:00, 499797.90it/s]
Correct: 25.0 out of 500.0: 5.0%
PS C:\Users\micah\OneDrive\Gmail OneDrive\Desktop\CMU\_Spring 2025\Deep Learning\HW
5\true_student>
```

(2e) Define a span corruption function for pretraining.

Sample examples from CharCorruptionDataset on the pretraining dataset wiki.txt:

```
PS C:\Users\micah\OneDrive\Gmail OneDrive\Desktop\CMU\_Spring 2025\Deep Learning\HW5\true_student> python src/dataset.py charcorruption
data has 418351 characters, 256 unique.
x: Khatchig⁇an⁇ Mouradi□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
y: acob H⁇ Studer. Jacob Henry Studer (26 February 1840 Colum⁇enry□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
x: John Stephen⁇w, Stephen became a welder's apprent⁇. Born in Glasgo□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
y: ohn Stephen⁇w, Stephen became a welder's apprent⁇. Born in Glasgo□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
x: Ge⁇gin⁇or□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
y: e⁇gin⁇or□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
PS C:\Users\micah\OneDrive\Gmail OneDrive\Desktop\CMU\_Spring 2025\Deep Learning\HW5\true_student>
```

(2f) Define a span corruption function for pretraining.

Dev set Accuracy:

```
Model on device:  cuda:0
/content/drive/MyDrive/student/src/run.py:329: FutureWarning: You are using `torch.load` w
  model.load_state_dict(torch.load(args.reading_params_path))
500it [00:47, 10.47it/s]
Correct: 118.0 out of 500.0: 23.599999999999998%
2025-04-09 06:56:47.563343: I tensorflow/core/util/port.cc:153] oneDNN custom operations a
2025-04-09 06:56:47.581858: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477]
```

The test set predictions are also saved. I am assuming that the TAs will run code to calculate the accuracy of this.

(2g) Write and try out a different kind of position embeddings (Budget about 1 hour for training)

Questions 2gi and 2gii are on latex.

Question 2giii

iii. (7 points) In the provided code, RoPE is implemented using the functions precompute rotary emb and apply rotary emb in src/attention.py. You need to implement these functions and the parts of code marked [part g] in src/attention.py and src/run.py to use RoPE in the model. Train a model with RoPE on the span corruption task and finetune it on the birthplace prediction task. Specifically, you should be able to run the following four commands:

Dev Set Accuracy:

```
data has 418351 characters, 256 unique.
number of parameters: 3290624
Model on device:  cuda:0
/content/drive/MyDrive/student/src/run.py:329: FutureWarning: You
  model.load_state_dict(torch.load(args.reading_params_path))
500it [00:58,  8.48it/s]
Correct: 169.0 out of 500.0: 33.800000000000004%
2025-04-09 06:40:54.168807: I tensorflow/core/util/port.cc:153] or
2025-04-09 06:40:54.186780: E external/local_xla/xla/stream_execut
WARNING: All log messages before absl::InitializeLog() is called a
E0000 00:00:1744180854.208598    5812 cuda_dnn.cc:8310] Unable to
```

The test set predictions are also saved. I am assuming that the TAs will run code to calculate the accuracy of this.

3. Considerations in pre-trained knowledge (5 points)

(a) (1 point) Succinctly explain why the pretrained (vanilla) model was able to achieve an accuracy of above 10%, whereas the non-pretrained model was not.

The pretrained model had prior exposure to a large corpus (Wikipedia), allowing it to learn general language structure and factual associations. In contrast, the non-pretrained model had to learn everything from scratch using limited task-specific data, which was insufficient for meaningful learning.

(b) (2 points) Take a look at some of the correct predictions of the pretrain+finetuned vanilla model, as well as some of the errors. We think you'll find that it's impossible to tell, just looking at the output, whether the model retrieved the correct birth place, or made up an incorrect birth place. Consider the implications of this for user-facing systems that involve pretrained NLP components. Come up with two distinct reasons why this model behavior (i.e. unable to tell whether it's retrieved or made up) may cause concern for such applications, and an example for each reason.

The fact that a pretrained model can just hallucinate incorrect information has very bad implications for user-facing systems.

1) **<u>Trust & Safety Risk:</u>**
   The model can hallucinate convincing but incorrect answers, and users may mistakenly trust them. For example, let's say a doctor needs to diagnose a patient and is using a healthcare chatbot to access the patient's patient-specific information and prepare a diagnosis. If the doctor were to trust the chatbot blindly, an incorrect diagnosis or incorrect dosage due to the model's hallucination would potentially put the patient in danger.
2) **<u>Debugging & Transparency Issues:</u>**
   When pipelines are large and downstream tasks rely on outputs from the chatbot model, hallucinations make it hard to trace errors. For example, in a neuroscience lab, researchers use a pipeline where a pre-trained language model helps annotate rodent genetic sequences by describing gene functions. If the model hallucinates a plausible but incorrect gene function, the error may go unnoticed until later experiments fail. Because the output appears scientifically valid, identifying the hallucination as the root cause becomes extremely difficult, slowing progress and wasting lab resources.

(c) (2 points) If your model didn't see a person's name at pretraining time, and that person was not seen at fine-tuning time either, it is not possible for it to have "learned" where they lived. Yet, your model will produce something as a predicted birth place for that person's name if asked. Concisely describe a strategy your model might take for predicting a birth place for that

person's name, and one reason why this should cause concern for the use of such applications. (While 3b discussed the problems that could arise from made up predictions, 3c asks for a mechanism the model could be using for generating birth places of people not seen at fine-tuning time and why such a mechanism could be problematic.)

While it's hard to know exactly what is going on under the hood, the model is likely making educated guesses in the same way a human would, leveraging learned patterns and similarities in the name structure of words. For example, names that sound German may be associated with "Berlin," and the model could guess this as their birth place. The learned patterns and similarities are a byproduct of how deep the model's attention mechanisms can learn patterns between tokens/words.

A huge concern for how the model is taking educated guesses is this can often come up as just straight up stereotyping. In the context of this model, the model may stereotype a person whose name is Jose as coming from Mexico City. This is an instance where the model's hallucination would exacerbate negative stereotypes. Outside of birthplace prediction, the negative stereotypes could be much worse, especially for a full-on chatbot.

1a. Permuting the input

i. Show that the output $Z_{\mathbf{perm}}$ for the permuted input $X_{\mathbf{perm}}$ will be $Z_{\mathbf{perm}} = PZ$

$$Z = \text{ReLU}(HW_1 + 1 \cdot b_1)W_2 + 1 \cdot b_2 \quad \text{and} \quad H = \text{softmax}(QK^T/\sqrt{d})V$$

Approaching this problem step by step:

1. The Query, Key, and Value Matrices are all affected by permuting X with the following transformation:
   - $Q_{\text{perm}} = PXW_Q$
   - $K_{\text{perm}} = PXW_K$
   - $V_{\text{perm}} = PXW_V$
2. The resulting H matrix, the output of multiplying the attention matrix by the value matrix, is changed resulting in the following transformation:
   - $H = \text{softmax}((PXW_Q)(PXW_K)^T/\sqrt{d})(PXW_V)$
3. Taking advantage of the following property, $\text{softmax}(PAP^T) = P\text{softmax}(A)P^T$, where P represents a permutation matrix, we can simplify the H matrix further:

   $$H = \text{softmax}(PXW_QW_Q^TX^TP^TW_K^T/\sqrt{d})(PXW_V)$$

   $$\text{softmax}(PAP^T) = P\text{softmax}(A)P^T, \text{ where } P = P, A = XW_QW_K^TX^T$$

   - The dimensions work out since A dimensions in order are:
     - $T \times d$
     - $d \times d$
     - $d \times d$
     - $d \times T$
4. Then:

   $$P \cdot \text{softmax}(A) \cdot P^T = P \cdot H \cdot P^T$$

   $$H = P \cdot \text{softmax}(A) \cdot P^T = P \cdot H \cdot P^T \Rightarrow H = PH \quad \text{since } P^T \text{ and } P \text{ cancel out to identity}$$

   $$Z = \text{ReLU}(PHW_1 + 1 \cdot b_1)W_2 + 1 \cdot b_2$$

5. We can now use the property $\text{ReLU}(PA) = P \cdot \text{ReLU}(A)$, letting $H^*W_1 + 1^*b_1 = A$, with $\text{ReLU}(P(HW_1 + 1^*b_1)) = P \cdot \text{ReLU}(A)$. This is possible because we can actually apply a sneaky trick where we set $1^*b_1 = P \cdot 1 \cdot b_1$. This is because permuting the ones does affect the end result at all, allowing us to factor out P from the expression $P \cdot H \cdot P^TW_1 + P \cdot 1 \cdot b_1$.
   - Now, $\text{ReLU}(P(A)) = P \cdot \text{ReLU}(A)$, which now allows us to simplify our expression to $Z = P \cdot \text{ReLU}(HW_1 + 1 \cdot b_1)W_2 + 1 \cdot b_2$

- Using the same sneaky trick from before, we can factor out P again with:
$$Z = P \cdot \text{ReLU}(HW_1 + 1 \cdot b_1)W_2 + P \cdot 1 \cdot b_2$$
- This means that the output Z_{perm} for any permuted input X_{perm} will be:
$$Z_{\text{perm}} = PZ$$

2. Pretrained Transformer models and knowledge access: Problem 2g Part i

To show equation 2 gives the same result as equation 1, we need to break down the element-wise multiplication and evaluate the matrix one pair of features at a time.

Take the i-th pair: $(x_t^{(2i-1)}, x_t^{(2i)})$

$$z_i = x_t^{(2i-1)} + i x_t^{(2i)}$$

Rotation is encoded as:

$$r_i = \cos(t\theta_i) + i \sin(t\theta_i)$$

Multiply the rotation and features element-wise:

$$r_i z_i = (\cos(t\theta_i) + i \sin(t\theta_i))(x_t^{(2i-1)} + i x_t^{(2i)})$$

FOIL out the product:

$$r_i z_i = x_t^{(2i-1)} \cos(t\theta_i) - x_t^{(2i)} \sin(t\theta_i) + i \cdot \left(x_t^{(2i-1)} \sin(t\theta_i) + x_t^{(2i)} \cos(t\theta_i) \right)$$

Real part $\Rightarrow$ rotated x-coordinate
Imaginary part $\Rightarrow$ rotated y-coordinate
The first part is the real component, which becomes the rotated x-coordinate.
The second part is the imaginary component, which becomes the rotated y-coordinate

$$\begin{bmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{bmatrix} \begin{bmatrix} x_t^{(2i-1)} \\ x_t^{(2i)} \end{bmatrix} = \begin{bmatrix} x_t^{(2i-1)} \cos(t\theta_i) - x_t^{(2i)} \sin(t\theta_i) \\ x_t^{(2i-1)} \sin(t\theta_i) + x_t^{(2i)} \cos(t\theta_i) \end{bmatrix}$$

Same result. Equation 2 just uses complex numbers instead of matrices.

2. Pretrained Transformer models and knowledge access: Problem 2g Part ii

To show the dot product depends only on relative position, we'll write both RoPE embeddings in complex form and simplify.

We use Euler's formula to encode rotation:

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

This turns 2D rotation into simple multiplication:

$$\text{RoPE}(z, t) = z \cdot e^{it\theta}$$

Now we want the dot product between two RoPE-embedded vectors:

$$\langle \text{RoPE}(z_1, t_1), \text{RoPE}(z_2, t_2) \rangle$$

Since these are complex numbers, we take the real part of the product with the conjugate:

$$= \text{Re}(z_1 \cdot e^{it_1\theta} \cdot \overline{z_2 \cdot e^{it_2\theta}})$$

This becomes:

$$= \text{Re}(z_1 \cdot \overline{z_2} \cdot e^{i(t_1 - t_2)\theta})$$

We conjugate z_2 to make the result a real number. Otherwise, the dot product of complex numbers wouldn't be as informative.

Now, regroup the expression:

$$= \text{Re}\left((z_1 \cdot e^{i(t_1 - t_2)\theta}) \cdot \overline{z_2}\right)$$

This is just:

$$= \langle \text{RoPE}(z_1, t_1 - t_2), \text{RoPE}(z_2, 0) \rangle$$

Therefore, the final result depends only on the difference $t_1 - t_2$.